Ancre — Plan de conception

Nov 2, 2026 · @Saussice

Vision et objectifs

Ancre est une application web personnelle, hébergée sur ton domaine Cloudflare, qui transforme tes cours (PDF ou DOCX) en questions de révision et te les fait réviser par sessions courtes, sans distraction. Elle n'a qu'un seul utilisateur : toi.

Trois principes guident toutes les décisions :

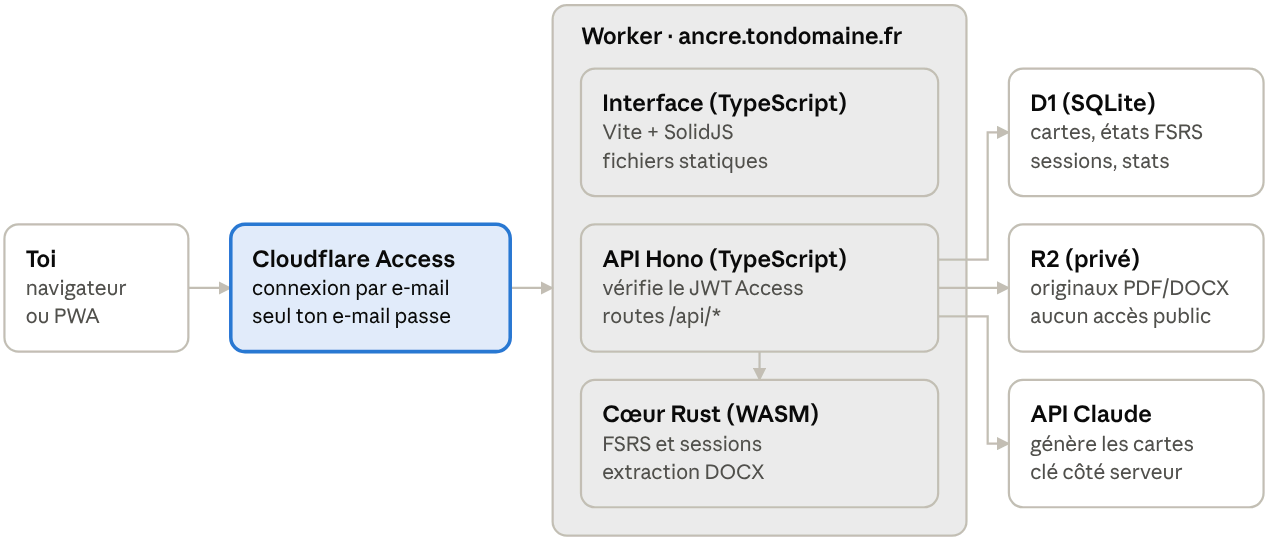
- **Zéro décision pendant la révision** : l'app choisit la prochaine carte, la durée et le moment de la pause.
- **Zéro friction à l'import** : on dépose un fichier, on relit les questions proposées, on valide.
- **Zéro exposition** : aucune route n'est accessible sans ton identité, et la clé de l'API Claude ne quitte jamais le serveur.

Le MVP couvre quatre fonctions : gérer des matières, importer un PDF/DOCX pour générer des cartes, réviser en mode focus avec répétition espacée (FSRS), et suivre sa progression.

Architecture globale

Tout tient dans un seul Worker Cloudflare, placé derrière Cloudflare Access : il sert l'interface, expose l'API et embarque le cœur Rust compilé en WebAssembly.

Toute requête franchit Cloudflare Access avant d'atteindre ton code



architecture · un Worker derrière Access

Le navigateur ne parle qu'à ton domaine ; seule l'API du Worker touche D1, R2 et Claude.

Pourquoi ce partage TypeScript / Rust. TypeScript gère ce qui est natif à l'environnement Workers : HTTP avec Hono, vérification JWT avec `jose`, appels à l'API Claude, liaisons D1 et R2. Rust porte la logique pure et testable avec `cargo test` : planification FSRS, composition des sessions, extraction DOCX, validation des cartes. Ce cœur pourra aussi servir plus tard dans une version bureau avec Tauri.

Sécurité : un seul utilisateur, toi

L'accès repose sur trois verrous indépendants : Cloudflare Access bloque tout visiteur avant le Worker, le Worker revérifie l'identité à chaque requête, et la clé Claude reste un secret côté serveur. Si l'un des verrous saute, les deux autres tiennent.

1. Cloudflare Access (Zero Trust) devant le domaine. Créer une application « self-hosted » sur `ancre.tondomaine.fr` avec une seule règle *Allow* : ton adresse e-mail. Méthode de connexion : code à usage unique par e-mail, ou Google/GitHub. Tout autre visiteur voit l'écran de connexion Cloudflare et n'atteint jamais ton code.

2. Vérification du JWT dans le Worker. Access ajoute un jeton signé dans l'en-tête `Cf-Access-Jwt-Assertion`. Cloudflare recommande de le valider dans le Worker même derrière Access, et de préférer cet en-tête au cookie `CF_Authorization`. Un middleware vérifie, sur chaque route `/api/*` :

- la signature RS256 avec les clés publiques de `https://<équipe>.cloudflareaccess.com/cdn-cgi/access/certs` (bibliothèque `jose`) ;
- `aud` = le tag AUD de l'application (variable `POLICY_AUD`) et `iss` = ton domaine d'équipe (`TEAM_DOMAIN`) ;
- `email` = `OWNER_EMAIL`, sinon réponse 403.

3. Fermer les portes dérobées. Dans `wrangler.toml` : `workers_dev = false` et `preview_urls = false`, pour que le Worker ne soit joignable que par ton domaine protégé. Le bucket R2 reste privé (aucun domaine public).

4. La clé API Claude. Stockée avec `wrangler secret put ANTHROPIC_API_KEY`, lue uniquement par le Worker, jamais envoyée au navigateur. Dans la console Claude, créer une clé dédiée à Ancre et fixer une limite de dépense mensuelle.

5. Garde-fous sur l'import. Taille maximale de 20 Mo, contrôle des octets magiques (`%PDF` pour un PDF, `PK` pour un DOCX), et un quota d'imports par jour stocké dans D1. Même en cas de fuite de session, la facture reste plafonnée.

Import PDF/DOCX et génération des questions

Un fichier déposé devient une liste de cartes « brouillon » que tu relis avant qu'elles entrent en révision. Claude lit les PDF nativement (texte et images de chaque page) ; un DOCX doit d'abord être converti en texte, car l'API ne l'accepte pas tel quel.

1. **Dépôt.** Dans l'écran Import, tu choisis la matière, le nombre de cartes souhaité et les types (question/réponse, texte à trous, QCM, « explique »). Le navigateur envoie le fichier en `multipart` à `POST /api/imports`.
2. **Contrôle et stockage.** Le Worker vérifie taille et octets magiques, range l'original dans R2 (`imports/<id>.pdf`) et crée une ligne `sources` au statut `processing`.
3. **Préparation.**
   - PDF : envoyé en bloc `document` base64. Limites actuelles de l'API : 32 Mo par requête et 600 pages (100 pour les modèles à fenêtre de 200k tokens). Au-delà, on importe chapitre par chapitre.
   - DOCX : le cœur Rust ouvre l'archive ZIP, lit `word/document.xml` et produit un Markdown qui conserve titres et listes. Ce texte part dans un bloc `text`.

4. **Appel à Claude.** Le Worker appelle `POST https://api.anthropic.com/v1/messages` avec le modèle défini dans la variable `CLAUDE_MODEL` (par défaut `claude-sonnet-5-5`). Pour obtenir un JSON fiable, on déclare un outil `save_cards` avec un schéma JSON et on force son usage via `tool_choice`. Chaque carte contient : `type`, `question`, `answer`, `choices` (QCM), `explanation`, `source_page`, `difficulty`.
5. **Validation.** Le cœur Rust contrôle le schéma, retire les doublons et les cartes trop longues, puis enregistre les cartes au statut `draft`.
6. **Relecture.** Tu acceptes, modifies ou supprimes chaque carte. Seules les cartes acceptées reçoivent un état FSRS et entrent dans les sessions.

Consignes du prompt système. Une notion par carte ; formuler en français ; privilégier le rappel actif plutôt que la reconnaissance ; couvrir toutes les notions clés du document ; citer la page source ; ne rien inventer qui ne soit pas dans le document.

Durée et coût. Un gros PDF peut prendre une minute. Pour le MVP, la requête reste ouverte et le Worker relaie le flux de Claude (streaming) pour afficher la progression. Plus tard, Cloudflare Queues permettra un traitement en arrière-plan. Chaque page de PDF consomme environ 1 500 à 3 000 tokens d'entrée, au tarif standard de l'API.

Modèle de données (D1)

Sept tables suffisent : les matières, les documents importés, les cartes, leur état FSRS, l'historique des réponses, les sessions et le parking à pensées. Les dates sont stockées en ISO 8601 (texte), les identifiants en UUID.

```
CREATE TABLE subjects (
  id          TEXT PRIMARY KEY,
  name        TEXT NOT NULL,
  color       TEXT,
  created_at  TEXT NOT NULL
);

CREATE TABLE sources (
  id          TEXT PRIMARY KEY,
  subject_id  TEXT NOT NULL REFERENCES subjects(id) ON DELETE CASCADE,
  filename    TEXT NOT NULL,
  kind        TEXT NOT NULL CHECK (kind IN ('pdf','docx')),
  r2_key      TEXT NOT NULL,
  status      TEXT NOT NULL CHECK (status IN
('processing','ready','failed')),
```

```
  tokens_in   INTEGER,
  tokens_out  INTEGER,
  error       TEXT,
  created_at  TEXT NOT NULL
);

CREATE TABLE cards (
  id           TEXT PRIMARY KEY,
  subject_id   TEXT NOT NULL REFERENCES subjects(id) ON DELETE CASCADE,
  source_id    TEXT REFERENCES sources(id) ON DELETE SET NULL,
  type         TEXT NOT NULL CHECK (type IN ('qa','cloze','mcq','explain')),
  question     TEXT NOT NULL,
  answer       TEXT NOT NULL,
  choices      TEXT,            -- JSON pour les QCM
  explanation  TEXT,
  source_page  INTEGER,
  status       TEXT NOT NULL CHECK (status IN
('draft','active','suspended')),
  created_at   TEXT NOT NULL
);

CREATE TABLE card_state (        -- état FSRS, une ligne par carte active
  card_id         TEXT PRIMARY KEY REFERENCES cards(id) ON DELETE CASCADE,
  due             TEXT NOT NULL,
  stability       REAL NOT NULL,
  difficulty      REAL NOT NULL,
  reps            INTEGER NOT NULL DEFAULT 0,
  lapses          INTEGER NOT NULL DEFAULT 0,
  state           TEXT NOT NULL CHECK (state IN
('new','learning','review','relearning')),
  last_review     TEXT
);
CREATE INDEX idx_card_state_due ON card_state(due);

CREATE TABLE reviews (           -- historique, utile pour les stats et
l'optimisation FSRS
  id           TEXT PRIMARY KEY,
  card_id      TEXT NOT NULL REFERENCES cards(id) ON DELETE CASCADE,
  session_id   TEXT REFERENCES sessions(id),
  grade        INTEGER NOT NULL CHECK (grade BETWEEN 1 AND 4),
  duration_ms  INTEGER,
  reviewed_at  TEXT NOT NULL
);
```

```
CREATE TABLE sessions (
  id            TEXT PRIMARY KEY,
  mode          TEXT NOT NULL CHECK (mode IN ('quick5','timed','subject')),
  planned_min   INTEGER,
  started_at    TEXT NOT NULL,
  ended_at      TEXT,
  cards_done    INTEGER NOT NULL DEFAULT 0
);

CREATE TABLE parked_thoughts (
  id          TEXT PRIMARY KEY,
  session_id  TEXT REFERENCES sessions(id),
  text        TEXT NOT NULL,
  done        INTEGER NOT NULL DEFAULT 0,
  created_at  TEXT NOT NULL
);
```

Les migrations vivent dans `migrations/` et s'appliquent avec `wrangler d1 migrations apply ancre --remote`.

API HTTP et répétition espacée

Toutes les routes passent par le middleware d'authentification de la section Sécurité et répondent en JSON.

Méthode	Route	Rôle
GET	`/api/subjects`	Lister les matières avec le nombre de cartes dues
POST	`/api/subjects`	Créer une matière
POST	`/api/imports`	Déposer un PDF/DOCX et lancer la génération (réponse en streaming)
GET	`/api/imports/:id`	Statut d'un import et cartes brouillon associées
PATCH	`/api/cards/:id`	Modifier une carte, l'accepter (`active`) ou la suspendre
DELETE	`/api/cards/:id`	Supprimer une carte

Méthode	Route	Rôle
POST	`/api/sessions`	Ouvrir une session (`quick5` , `timed` ou `subject`) et recevoir la file de cartes
POST	`/api/sessions/:id/reviews`	Enregistrer une réponse (note 1 à 4) et recalculer l'échéance
POST	`/api/sessions/:id/end`	Clôturer la session et renvoyer le bilan
POST	`/api/parked`	Noter une pensée parasite
GET	`/api/stats`	Progression par matière et temps de révision

FSRS dans le cœur Rust. Après chaque réponse, tu donnes une note : 1 « à revoir », 2 « difficile », 3 « bien », 4 « facile ». Le cœur expose une fonction pure `schedule(state, grade, now) -> state` qui calcule la nouvelle stabilité, la difficulté et la prochaine date. Cible de rétention : 90 %. Candidats : la crate `rs-fsrs` (planificateur seul, léger) ou `fsrs` (projet officiel, plus lourd car il inclut l'optimisation) ; vérifier qu'elle compile en `wasm32-unknown-unknown` .

Composition d'une session. Le cœur trie les cartes dues par date, limite les cartes nouvelles (par défaut 10 par jour), puis alterne les matières et les types de questions pour éviter la monotonie. En mode `quick5` , il prend les 5 cartes les plus en retard.

Interface et mode focus

L'écran d'accueil propose une seule action principale, « Réviser 5 cartes » ; tout le reste est secondaire. L'interface est une PWA installable, utilisable au clavier sur ordinateur et au doigt sur téléphone.

Écran	Contenu
Accueil	Gros bouton « 5 cartes », bouton « Session 15 min », cartes dues par matière
Import	Dépôt du fichier, choix de la matière, du nombre et des types de cartes, progression en direct
Relecture	Cartes brouillon une par une : accepter, modifier, supprimer

Écran	Contenu
Session focus	Une carte en plein écran, champ de réponse, barre de progression « 4 / 12 », rien d'autre
Pause	Minuteur de pause, suggestion de bouger ou boire, bouton « Reprendre » ou « Arrêter là »
Fin de session	Bilan (cartes faites, temps), liste du parking à pensées
Matières	Liste, cartes par matière, recherche, édition
Statistiques	Cartes maîtrisées, à venir, temps réel de révision par semaine

Règles du mode focus.

- Pas de navigation visible pendant la session ; seule la touche `Échap` met en pause.
- Réponse tapée avant affichage de la correction (rappel actif), puis notation 1 à 4.
- Raccourcis : `Espace` révéler, `1` – `4` noter, `P` ouvrir le parking à pensées, `Échap` pause.
- Durée de session réglable (10 à 25 min), pause proposée automatiquement à la fin.
- Animations douces, couleurs calmes, thème clair et sombre ; aucune notification pendant la session.
- Pas de série (streak) punitive : le tableau de bord montre ce qui est acquis, pas ce qui a été manqué.

Stack frontend. TypeScript avec Vite et SolidJS (ou Svelte), styles en CSS simple ou Tailwind. Le build statique est servi par le même Worker grâce aux *static assets*.

Structure du dépôt, déploiement et coûts

Un monorepo avec trois paquets : le cœur Rust, le Worker TypeScript et le frontend. Un seul `wrangler deploy` publie l'API et l'interface sur ton domaine.

```
ancre/
├── core/                  # crate Rust → WebAssembly (wasm-bindgen)
│   ├── src/fsrs.rs        # planification
│   ├── src/session.rs     # composition des sessions
│   ├── src/docx.rs        # extraction DOCX → Markdown
│   └── src/cards.rs       # validation et dédoublonnage
├── worker/                # Worker TypeScript (Hono)
│   ├── src/auth.ts        # vérification JWT Access
│   ├── src/claude.ts      # appel API Claude + outil save_cards
│   ├── src/routes/        # subjects, imports, cards, sessions, stats
│   ├── migrations/        # schéma D1
│   └── wrangler.toml
└── web/                   # frontend Vite + SolidJS
    └── src/
```

Étapes de déploiement.

1. Créer la base et le bucket : `wrangler d1 create ancre`, `wrangler r2 bucket create ancre-imports`.
2. Déclarer dans `wrangler.toml` les liaisons D1 et R2, les *static assets* (`web/dist`), la route personnalisée `ancre.tondomaine.fr`, `workers_dev = false` et `preview_urls = false`.
3. Ajouter les secrets et variables : `ANTHROPIC_API_KEY` (secret), `CLAUDE_MODEL`, `TEAM_DOMAIN`, `POLICY_AUD`, `OWNER_EMAIL`.
4. Dans Zero Trust, créer l'application Access sur le sous-domaine avec la règle limitée à ton e-mail, puis copier son tag AUD dans `POLICY_AUD`.
5. Construire : `wasm-pack build core --target bundler`, puis le frontend, puis `wrangler deploy`.
6. Tester depuis une fenêtre de navigation privée : l'écran de connexion Access doit apparaître, et un appel `curl` direct à `/api/subjects` doit être refusé.

Coûts. Pour un usage personnel, les paliers gratuits de Workers, D1, R2 et Access suffisent. Le seul coût variable est l'API Claude, facturée au token et plafonnée par ta limite de dépense. Point de vigilance : le palier gratuit de Workers limite le temps CPU par requête ; si l'extraction d'un gros DOCX dépasse, passer au forfait Workers Paid (5 $/mois).

Feuille de route

Le projet avance en quatre phases ; chacune se termine sur quelque chose d'utilisable.

1. **Fondations sécurisées.** Worker « hello » sur ton domaine, Access configuré, middleware JWT, D1 et migrations. *Terminé quand* une requête hors connexion est refusée et que la page s'affiche après connexion.
2. **Révision manuelle.** Cœur Rust (FSRS + sessions) compilé en WASM, CRUD des matières et cartes, mode focus, minuteur, parking à pensées. *Terminé quand* tu révises chaque jour des cartes écrites à la main.
3. **Import intelligent.** Dépôt PDF/DOCX, stockage R2, extraction DOCX en Rust, appel Claude avec l'outil `save_cards`, écran de relecture, quotas. *Terminé quand* un cours de 20 pages donne des cartes exploitables en moins de deux minutes.
4. **Confort et suivi.** Statistiques, PWA hors ligne pour réviser sans réseau, import par chapitres, file d'attente Cloudflare Queues pour les gros documents, export des cartes.

Questions ouvertes. Quel framework frontend préfères-tu (SolidJS ou Svelte) ? Veux-tu aussi générer des cartes à partir d'images (photos de tableau ou de notes manuscrites) ?

Sources

- Cloudflare — Validate JWTs (Access)
- Claude — PDF support